

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PASHCHIMANCHAL CAMPUS

A Major Project Proposal
On

A Multi-Agent Reinforcement Learning Framework for Adaptive Traffic Signal Control, Congestion Mitigation, and Emergency Vehicle Prioritization

Submitted By:

Aashish Adhikari (PAS079BCT003)
Ashim Shrestha (PAS079BCT007)
Sagun Paudel (PAS079BCT032)
Saugat Adhikari (PAS079BCT036)

Submitted To:

Department of Electronics and Computer Engineering
Pashchimanchal Campus
Pokhara, Nepal

June 1, 2026

TABLE OF CONTENTS

LIST OF FIGURES iii

LIST OF TABLES iv

LIST OF ABBREVIATIONS iv

1 INTRODUCTION 1

1.1 Background 1
1.2 Problem statement 1
1.3 Objectives 2

2 LITERATURE REVIEW 3

2.1 Existing Approaches to Intersection Control and Congestion Management 3
2.2 Review of Existing Digital and Simulated Traffic Management Systems 3
2.3 Multi-Agent Coordination Paradigms and Algorithm Selection 4
2.4 Research Gap and Relevance of the Proposed System 5

3 PROJECT FEASIBILITY 6

3.1 Operational Feasibility 6
3.2 Technical Feasibility 6
3.3 Social Feasibility 7
3.4 Legal and Ethical Feasibility 7
3.5 Economic Feasibility 8

4 METHODOLOGY 9

4.1 Requirements Gathering 9
4.2 Design 10
4.2.1 UI/UX Design 10
4.2.2 System Design 10
4.3 Reinforcement Learning Formulation and Algorithm Selection 10
4.4 Development 12
4.5 Testing 12
4.6 Security, Monitoring, and Robustness Evaluation 13
4.7 Deployment / Simulation Run 13

5 REQUIREMENT ANALYSIS 15

5.1 Functional Requirements 15
5.1.1 User Authentication and Access Control 15

5.1.2 Simulation Control . . . 15
5.1.3 Traffic State Monitoring . . . 15
5.1.4 Agent Decision-Making . . . 15
5.1.5 Signal Phase Updates . . . 16
5.1.6 Emergency Vehicle Priority . . . 16
5.1.7 Dashboard Reporting . . . 16
5.1.8 Experiment Comparison . . . 16
5.1.9 Secure Logging, Monitoring, and Resilience . . . 16
5.2 Non-Functional Requirements . . . 17
5.3 Risk Analysis . . . 18
5.4 Expected Deliverables . . . 18

6 Diagrams **20**
6.1 Use Case Diagram . . . 20
6.2 Data Flow Diagram . . . 21
6.2.1 DFD Level 0 . . . 21
6.2.2 DFD Level 1 . . . 21
6.3 System Architecture Diagram . . . 22
6.4 Gantt Chart . . . 23

7 Tools and Technologies **25**
7.1 Frontend Infrastructure . . . 25
7.2 Backend Infrastructure . . . 25
7.3 Simulation Tools . . . 26
7.4 AI/ML Tools . . . 26
7.5 APIs and Data Sources . . . 26
7.6 Hosting and Deployment . . . 27
7.7 Security and Monitoring Infrastructure . . . 27
7.8 Testing Tools . . . 27
7.9 Version Control . . . 27

8 Conclusion **29**

REFERENCES **30**

List of Figures

4.1 Agile Model 9
6.1 Use Case Diagram for the MARL Traffic Control System 20
6.2 Data Flow Diagram Level 0 for the MARL Traffic Control System 21
6.3 Data Flow Diagram Level 1 for the MARL Traffic Control System 22
6.4 System Architecture Diagram 23
6.5 Gantt Chart 24

List of Tables

4.1 Experiment matrix for evaluation scenarios (S1–S5). 13
5.1 Risk analysis with likelihood, impact, and mitigation. 18

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CI/CD	Continuous Integration / Continuous Deployment
CTCE	Centralized Training with Centralized Execution
CTDE	Centralized Training with Decentralized Execution
Dec-POMDP	Decentralized Partially Observable Markov Decision Process
DTDE	Decentralized Training with Decentralized Execution
EPyMARL	Extended Python Multi-Agent Reinforcement Learning
EV	Emergency Vehicle
HTTPS	Hypertext Transfer Protocol Secure
MADDPG	Multi-Agent Deep Deterministic Policy Gradient
MAPPO	Multi-Agent Proximal Policy Optimization
MARL	Multi-Agent Reinforcement Learning
ML	Machine Learning
OWASP	Open Web Application Security Project
PyMARL	Python Multi-Agent Reinforcement Learning
QMIX	Q-Mixing Network (value-decomposition MARL algorithm)
RL	Reinforcement Learning
SB3	Stable-Baselines3
SCATS	Sydney Co-ordinated Adaptive Traffic System
SCOOT	Split Cycle Offset Optimization Technique
SPA	Single-Page Application
SQL	Structured Query Language
SUMO	Simulation of Urban MObility
TLS	Transport Layer Security
TraCI	Traffic Control Interface
VDN	Value Decomposition Network
ZAP	Zed Attack Proxy (OWASP ZAP)

1. INTRODUCTION

1.1. Background

Urban road traffic congestion is one of the most persistent challenges faced by rapidly growing cities. As vehicle numbers grow far faster than road infrastructure, intersections become the primary bottlenecks of the transportation network, directly affecting travel time, fuel consumption, and emissions. Traditionally, intersections are managed by fixed-time signal controllers whose phase durations are pre-computed offline from historical counts. Although simple and inexpensive, such controllers cannot respond to the dynamic and stochastic nature of real traffic demand.

Reinforcement Learning (RL) has emerged as a promising data-driven paradigm for signal control: by framing the controller as an agent that observes an intersection's state and learns, through trial and error, a policy minimizing a cost such as cumulative waiting time, RL adapts to traffic without a hand-designed flow model. When a network contains many interacting intersections, the problem extends to Multi-Agent Reinforcement Learning (MARL), in which each intersection is its own learning agent and the agents must coordinate for good network-wide performance. This project designs, implements, and evaluates a fully simulated, AI-driven urban mobility system built around MARL. Each signalized intersection is an autonomous agent in the open-source simulator SUMO, coupled to the learning back-end through TraCI, and the system adds an emergency vehicle prioritization mechanism and a web dashboard for visualization and experiment comparison. The proposed system adopts a cooperative multi-agent reinforcement learning approach like QMIX value-decomposition algorithm, allowing agents to learn network-wide optimization strategies while operating on local observations.The project is a simulation-based research prototype rather than a deployed city-scale product.

1.2. Problem statement

Conventional traffic signal control strategies suffer from several well-documented limitations. Fixed-time controllers cannot react to real-time fluctuations in demand and frequently allocate green time to empty approaches while vehicles queue on congested ones. Rule-based actuated controllers, although reactive, depend on manually tuned thresholds that do not scale to networks with many interacting intersections and do not optimize for global, network-wide objectives. As a result, urban networks experience avoidable delays, long queues, reduced throughput, and increased emissions. A second, equally critical problem is the prioritization of emergency vehicles such as ambulances and fire en-

gines, which in most existing networks must navigate the same signal timings as ordinary traffic, leading to dangerous delays in time-critical situations. Therefore, there is a need for an adaptive, coordinated, and learning-based control system that can simultaneously reduce congestion and provide priority to emergency vehicles, while remaining feasible to develop and rigorously evaluate within a simulated environment using open-source tools.

1.3. Objectives

The main objective of this project is:

1. To implement and evaluate a SUMO-based MARL signal controller (QMIX) that targets reduction in average waiting time and emergency vehicle delay relative to fixed-time and rule-based baselines.

2. LITERATURE REVIEW

Traffic signal control has been studied extensively for several decades, and the approaches can broadly be grouped into fixed-time control, actuated/rule-based control, and learning-based control. The growing volume of urban traffic has made the efficient coordination of signalized intersections a problem of considerable practical and research importance. Because deploying experimental controllers on real road infrastructure is costly and risky, the majority of research in this domain is conducted in microscopic simulation, where individual vehicles and their interactions are modeled in a controllable and reproducible environment [1].

2.1. Existing Approaches to Intersection Control and Congestion Management

Fixed-time (pre-timed) controllers assign a static cycle length and a fixed sequence of phases to each intersection, with timings computed offline from historical average volumes. Although robust and computationally trivial, they are open-loop and cannot react to deviations from the assumed demand. Actuated and rule-based adaptive controllers, such as the widely deployed SCATS and SCOOT systems, introduce sensor feedback to extend or terminate green phases and coordinate plans across networks. However, these systems rely on hand-engineered heuristics and pre-defined plan libraries, and their performance is bounded by the quality of the rules chosen by engineers. Reinforcement Learning reframes signal control as a sequential decision-making problem in which an agent learns a policy that maximizes long-term cumulative reward; classical algorithms such as Q-learning have been applied successfully to single intersections [2], while Deep Reinforcement Learning has enabled controllers to handle high-dimensional state representations [3]. Multi-Agent Reinforcement Learning extends these ideas to networks of intersections, where each intersection is controlled by a separate agent, and foundational work has shown that multi-agent learning controllers, especially when augmented with communication, substantially outperform fixed and rule-based controllers as traffic load increases [4, 5].

2.2. Review of Existing Digital and Simulated Traffic Management Systems

SUMO (Simulation of Urban MObility) is an open-source, microscopic, multimodal traffic simulator that has become a de facto standard in the research community, supporting arbitrary road networks, demand definition, and special vehicle classes such as emergency vehicles [1]. Crucially, SUMO exposes the TraCI (Traffic Control Interface), which allows external programs to query the simulation state and control traffic lights at every step, making it ideal for closing the loop between a learning

agent and a traffic environment. Several research toolkits have been built on top of SUMO to expose intersections as standardized reinforcement learning environments, and the broader ecosystem of standardized interfaces such as Gymnasium and high-quality algorithm implementations has made it practical to apply deep RL without re-implementing algorithms from scratch.

2.3. Multi-Agent Coordination Paradigms and Algorithm Selection

A recent comprehensive survey of MARL in intelligent transportation systems [6] organizes the field along two axes: the coordination model, which governs how agents are trained and how they act, and the underlying learning algorithm. Three coordination models are commonly distinguished. In Centralized Training with Centralized Execution (CTCE), a single controller observes the global state and dictates the joint action, yielding strong coordination but scaling poorly and introducing a single point of failure. In Decentralized Training with Decentralized Execution (DTDE), each agent learns independently from its local observations, which is simple and scalable but suffers from non-stationarity, since every agent's environment is itself changing as the other agents learn. Centralized Training with Decentralized Execution (CTDE) occupies the middle ground: agents are trained with access to global information but, once trained, act using only their local observations. CTDE is the dominant paradigm in contemporary MARL and is particularly well suited to traffic signal control, where intersections must coordinate yet must ultimately act on locally available sensor data.

Within the CTDE paradigm, value-decomposition methods are a natural fit for traffic signal control because the action space, namely the set of admissible signal phases, is discrete. Value Decomposition Networks (VDN) approximate the joint action-value function as a simple sum of per-agent utilities [7], whereas QMIX generalizes this by combining the individual agent values through a learned, monotonic mixing network that is conditioned on the global state [8]. The monotonicity constraint preserves consistency between each agent's greedy action and the joint greedy action, which is precisely what allows decentralized execution to remain consistent with the centrally trained value function. The survey explicitly identifies QMIX as suitable for partially observable, cooperative tasks such as traffic signal control [6]. By contrast, actor-critic methods such as MADDPG are aimed primarily at continuous action spaces and mixed cooperative-competitive settings, while fully independent (DTDE) learners are vulnerable to the non-stationarity that arises in a coordinated network. For these reasons, the present project adopts QMIX as its primary learning algorithm and retains VDN as a simpler value-decomposition baseline for comparison.

2.4. Research Gap and Relevance of the Proposed System

Despite these advances, existing simulated systems often focus on either single-intersection control or congestion minimization alone and omit an integrated, user-facing visualization and experiment-management layer. A coherent prototype that combines multi-agent coordination, emergency vehicle prioritization, and an interactive dashboard in one reproducible system remains rare [9]. The proposed system addresses these gaps by adopting a MARL formulation in which each intersection learns and coordinates, by integrating an emergency prioritization objective of clear social importance, and by coupling the learning back-end to a web dashboard that exposes the simulation, the agents' behavior, and the evaluation metrics to the user. Security and robustness of the control system are also frequently overlooked; recent studies have shown that deep reinforcement learning traffic controllers are vulnerable to adversarial manipulation through falsified vehicle or sensor information [10], and that the surrounding control APIs constitute an attack surface in their own right. The proposed system therefore incorporates a security and observability layer, comprising secure logging and monitoring, API hardening, and an attack-simulation facility for evaluating policy robustness. The novelty of the project is the deliberate integration of multi-agent coordination, emergency prioritization, secure and observable operation, and reproducible experimentation into a single simulated research prototype that is feasible to build within the scope of a major project.

3. PROJECT FEASIBILITY

The proposed system is a fully simulated, AI-driven urban mobility platform that uses Multi-Agent Reinforcement Learning to control a network of signalized intersections for adaptive congestion mitigation and emergency vehicle prioritization. Because the project is conducted entirely in simulation using open-source tools, it is well suited to the scope and resources of a university-level major project. This feasibility study evaluates the operational, technical, social, legal, and economic feasibility of the proposed system.

3.1. Operational Feasibility

The project is operationally feasible because it is conducted entirely in simulation and does not require access to physical road infrastructure, municipal cooperation, or field deployment.

- The primary users are the project team and evaluators, who interact with the system through a web dashboard to configure simulations, launch training and evaluation runs, and inspect results.
- The operational workflow, designing a network, defining demand, training agents, running evaluations, and comparing experiments, is well understood and entirely within the control of the team.
- All components run on commodity hardware, so the system can be operated repeatedly and reproducibly without external dependencies.

3.2. Technical Feasibility

The project is technically feasible due to the availability of mature, well-documented, and freely available open-source technologies.

- SUMO provides the microscopic traffic simulation and the TraCI interface for programmatic, step-by-step control.
- Python serves as the integration language, while PyTorch and an established MARL framework (PyMARL/EPyMARL) provide robust implementations of the value-decomposition algorithms, with Gymnasium/PettingZoo for the environment interface and Stable-Baselines3 for the single-agent baseline.

- FastAPI provides a lightweight backend, and React with Tailwind CSS provides the interactive dashboard.
- Security and observability are equally well supported by open-source tooling: Prometheus and Grafana provide metrics collection and monitoring, while OWASP ZAP supports automated API security testing, all of which integrate readily with the FastAPI backend.
- The principal technical challenges, non-stationarity in multi-agent learning, reward shaping, and training stability, are mitigated by keeping the network size modest, using established algorithms, and structuring the work iteratively so a working single-agent baseline is achieved before scaling to the multi-agent setting.

3.3. Social Feasibility

The project has a clear and positive social dimension.

- Reduced congestion translates into lower travel times, reduced fuel consumption, and reduced emissions, all of which benefit urban communities.
- The emergency vehicle prioritization component directly addresses public safety by aiming to reduce the response time of ambulances and fire services.
- Because the system is a simulation, it carries no risk to the public during development, and its results and visualizations are intended to be accessible and interpretable for demonstration to a non-specialist audience.

3.4. Legal and Ethical Feasibility

As a purely simulated project that uses synthetic or openly licensed map data and does not collect personal data, the project raises minimal legal and ethical concerns.

- The open-source tools employed are distributed under permissive or copyleft licenses (such as EPL for SUMO and MIT/BSD-style licenses for the Python and JavaScript libraries) that permit academic use.
- Where road geometry is derived from OpenStreetMap, attribution requirements under the Open Database License are respected.

- The project is transparent about the fact that prioritizing emergency vehicles necessarily imposes some delay on other road users, and the evaluation quantifies this trade-off rather than obscuring it.
- Any adversarial or attack-simulation experiments are conducted exclusively against the project's own isolated simulation and API, constituting defensive security research that improves the robustness of the system and poses no risk to external parties.

3.5. Economic Feasibility

The project is economically feasible because it relies exclusively on free and open-source software and can be developed on the team members' existing computers.

- **Development Cost:** Minimal, as all core tools (SUMO, Python, PyTorch, FastAPI, React) are open-source and development is done in-house.
- **Compute Cost:** The baseline plan assumes development on standard laptops or desktops; where additional training compute is required, free or institutional cloud resources may be used.
- **Maintenance Cost:** Limited to occasional updates and bug fixes, as there is no production hosting requirement.

Since the platform is a research prototype and not intended for commercial deployment, the return on investment is measured in demonstrated congestion mitigation, improved coordination, and research value, making the cost fully justifiable.

4. METHODOLOGY

The development of the proposed system will follow the Agile method, which emphasizes iterative development, flexibility, and continuous feedback. Rather than building the complete system in a single pass, the work is divided into short iterations, each producing a working, testable increment. An early iteration establishes a single-intersection RL controller; subsequent iterations extend the system to multiple coordinated agents, add emergency vehicle prioritization, and integrate the dashboard. This approach is well suited to a research prototype, where design choices are refined in light of intermediate experimental results.

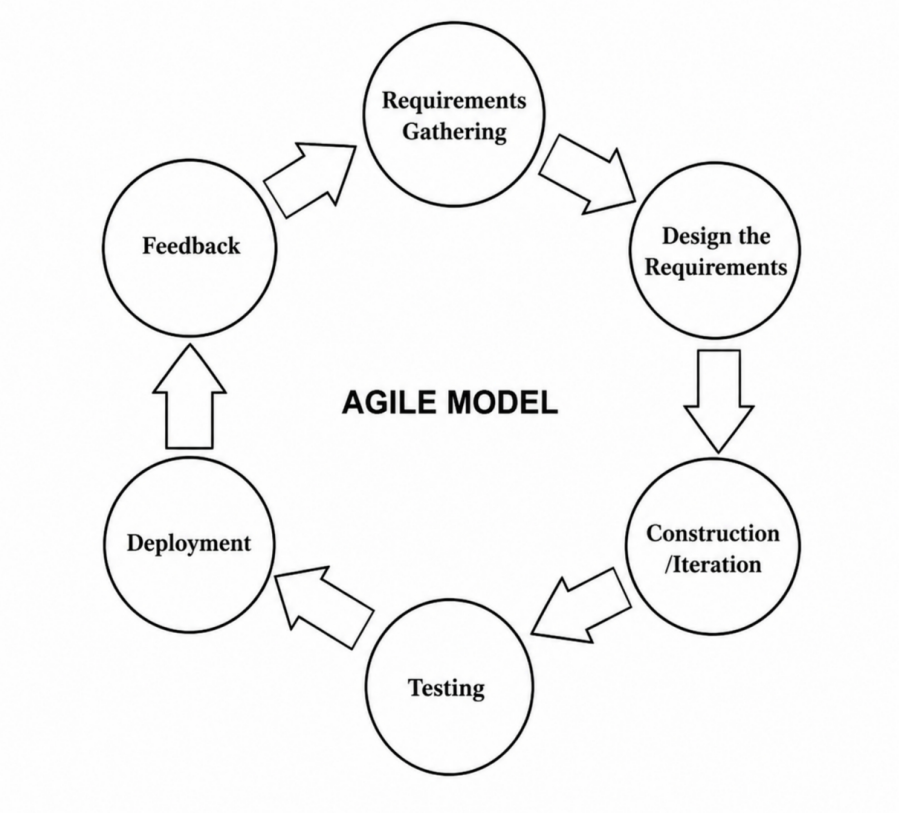


Figure 4.1: Agile Model

4.1. Requirements Gathering

The first phase involves eliciting functional and non-functional requirements through a study of the relevant literature, examination of existing SUMO-based RL toolkits, and discussion with the project supervisor. The outcomes of this phase are a clear specification of the simulated network, the agents' observation and action spaces, the reward formulation, the emergency prioritization behavior, and

the metrics and visualizations the dashboard must provide. The requirements are documented and prioritized so that the most essential capabilities, a functioning simulator-agent loop and a baseline comparison, are delivered first.

4.2. Design

There will be two main parts to the design phase:

4.2.1. UI/UX Design

The user interface is designed as a clean, single-page web dashboard. Wireframes and mockups are prepared for the control panel (selecting a network, configuring parameters, starting and stopping runs), the live monitoring view (rendering the network, signal phases, queues, and any active emergency vehicle), and the analytics view (charts of evaluation metrics and side-by-side experiment comparison). The design emphasizes clarity, responsiveness, and interpretability, using consistent color coding for signal states.

4.2.2. System Design

The system is designed as a modular, loosely coupled, four-layer architecture: the simulation layer (SUMO with the network, routes, and emergency vehicle definitions); the interface layer (a Python module communicating with SUMO through TraCI to extract observations and apply actions); the learning layer (the MARL framework built on Gymnasium/PettingZoo environment wrappers and PyTorch, hosting the per-intersection QMIX agents); and the application layer (a FastAPI backend and a React dashboard). Spanning the application layer is a cross-cutting security and observability concern, comprising authenticated APIs, structured logging, and a Prometheus/Grafana monitoring stack. Clear interfaces between layers ensure that, for example, the RL algorithm can be changed without modifying the dashboard.

4.3. Reinforcement Learning Formulation and Algorithm Selection

The traffic network is modeled as a decentralized partially observable Markov decision process (Dec-POMDP), in which each signalized intersection $i \in \{1, 2, \ldots, N\}$ is treated as a cooperative learning agent. The three components of each agent's decision problem are defined as follows:

- **Observation o_t^i:** for every incoming lane of intersection i, the queue length and accumulated waiting time, together with the current signal phase, the time elapsed in that phase, and a presence flag and normalized distance for any approaching emergency vehicle.
- **Action a_t^i:** selection of the next green phase from the set of admissible, collision-free phases for that intersection, subject to minimum-green and amber-time safety constraints.
- **Reward r_t:** a shared (team) reward that the agents jointly maximize, defined as the negative of the average accumulated waiting time across the network, normalized by N, with an additional penalty term that activates when an emergency vehicle is delayed:

$$r_t = -\frac{1}{N}\sum_{i=1}^{N}\sum_{l\in L_i} w_{i,l}(t) \;-\; \frac{\lambda}{N}\sum_{i=1}^{N} e_i(t) \tag{4.1}$$

where $w_{i,l}(t)$ is the waiting time on lane l of intersection i, L_i is the set of incoming lanes, $e_i(t)$ measures the delay imposed on any emergency vehicle at intersection i, and $\lambda > 0$ weights the priority given to emergency vehicles relative to overall congestion. The normalization by N keeps the reward scale stable across different network sizes, and λ is selected via a small grid search on validation scenarios to balance congestion reduction against emergency delay.

Following the analysis in Section 2.3, the project adopts **QMIX**, a value-decomposition algorithm operating under the centralized-training, decentralized-execution (CTDE) paradigm. During training, each agent maintains an individual action-value network $Q_i(o_t^i, a_t^i; \theta_i)$, and a mixing network combines these into a joint action-value function:

$$Q_{tot}(s_t, \mathbf{a}_t) = f_{\text{mix}}\big(Q_1, Q_2, \ldots, Q_N; s_t\big) \tag{4.2}$$

where s_t is the global state available only during training. The mixing network is constrained to be monotonic in each agent's value,

$$\frac{\partial Q_{tot}}{\partial Q_i} \geq 0, \quad \forall i, \tag{4.3}$$

which guarantees that maximizing each local Q_i also maximizes Q_{tot}. The network is trained by minimizing the temporal-difference loss $\mathscr{L} = \big(y_t - Q_{tot}(s_t, \mathbf{a}_t)\big)^2$ with target $y_t = r_t + \gamma \max_{\mathbf{a}'} Q_{tot}(s_{t+1}, \mathbf{a}'; \theta^-)$, where θ^- denotes a periodically updated target network and γ is the discount factor. Because of the

monotonicity constraint, at execution time each intersection simply acts greedily with respect to its own Q_i using only its local observation o_t^i, yielding fully decentralized control. QMIX is preferred over the simpler VDN (which assumes a purely additive decomposition) for its greater representational capacity, and over MADDPG and MAPPO because the signal-control action space is naturally discrete; VDN is nonetheless retained as a baseline. All approaches are evaluated against fixed-time and rule-based controllers using average waiting time, average queue length, throughput, and average travel time. Ablation studies will also evaluate the effect of removing the emergency penalty ($\lambda = 0$) and of substituting VDN for QMIX to isolate each component's contribution.

4.4. Development

The development phase proceeds in small sprints aligned with the prioritized requirements. The first sprint establishes the SUMO-TraCI loop and a single-agent RL controller validated against a fixed-time baseline. The second sprint generalizes the environment wrapper to multiple intersections and implements the multi-agent training scheme. The third sprint adds the emergency vehicle prioritization logic, and the fourth builds the FastAPI backend and React dashboard and connects them to the running simulation. The multi-agent learning components are developed in Python using PyTorch and a MARL framework (PyMARL/EPyMARL) with Gymnasium/PettingZoo environment wrappers, while Stable-Baselines3 is used only for the single-agent RL baseline; the frontend is developed using React and Tailwind CSS.

4.5. Testing

Testing is carried out continuously. Unit tests verify individual components such as observation extraction, reward computation, and action application, using tools such as Pytest. Integration tests verify that the simulator, interface, learning, and application layers operate correctly together. Experimental validation treats the trained agents as the object under test: the MARL controller is compared against fixed-time and rule-based baselines on identical scenarios using the metrics of average waiting time, average queue length, throughput, and average travel time. The evaluation scenarios are organized as the experiment matrix in Table 4.1.

Table 4.1: Experiment matrix for evaluation scenarios (S1–S5).

Scenario	Network size	Demand level	Emergency rate	Purpose
S1	Single intersection	Low	None	Sanity check and pipeline validation
S2	Single intersection	Medium	None	Baseline tuning and reward sanity
S3	Multi-intersection (4)	Medium	None	Coordination gains vs baselines
S4	Multi-intersection (4)	Medium	Low	Emergency prioritization trade-off study
S5	Multi-intersection (9)	High	Medium	Stress test for scalability and robustness

Ablation studies compare the full model against variants with $\lambda = 0$ and with VDN in place of QMIX to quantify the contribution of emergency prioritization and value decomposition. Reproducibility is ensured by fixing random seeds, recording configurations, and averaging results over multiple simulation runs.

4.6. Security, Monitoring, and Robustness Evaluation

Security and robustness are treated as a research contribution rather than an operational add-on. The prototype will harden its control API with OAuth2/JWT, input validation, rate limiting, and TLS, and will expose structured logs and metrics to Prometheus/Grafana so anomalies are measurable. A controlled attack-simulation harness, including OWASP ZAP scans and falsified vehicle or sensor inputs, will quantify how policy performance degrades under adversarial conditions within the isolated testbed. This evaluation makes the system's security posture and robustness explicit and reproducible rather than assumed.

4.7. Deployment / Simulation Run

Because the project is a simulation prototype, deployment refers to packaging the system so that a complete experiment can be launched reproducibly. The components are containerized using Docker and orchestrated with Docker Compose so that the simulator, learning back-end, web application, and the Prometheus/Grafana monitoring stack can be brought up consistently on any machine. In a

typical end-to-end run, the user opens the dashboard, selects a network and traffic scenario, and either loads a pre-trained policy or launches a training session; the backend drives SUMO through TraCI while the agents observe and act at each step; and the dashboard streams the evolving traffic state and metrics, which are stored for later comparison against other runs, with system health and security events monitored throughout.

5. REQUIREMENT ANALYSIS

5.1. Functional Requirements

5.1.1. User Authentication and Access Control

- The dashboard shall implement a lightweight, token-based authentication mechanism (OAuth2/JWT) so that saved experiments and configurations are associated with a user.
- Authentication is scoped to the prototype: a minimal local user store and role model is sufficient, and no external identity provider is required.
- Only authenticated users shall be permitted to start, stop, or reconfigure simulations and to access saved experiment data.

5.1.2. Simulation Control

- The system shall allow the user to select a road network and configure simulation parameters such as duration, demand level, and random seed.
- The system shall allow the user to start, pause, and stop simulation runs from the dashboard.
- The system shall support both training sessions and evaluation runs using pre-trained policies.

5.1.3. Traffic State Monitoring

- The system shall extract, at each simulation step, the relevant traffic state including queue lengths, waiting times, current signal phases, and vehicle positions.
- The system shall render this state in a live monitoring view of the network.

5.1.4. Agent Decision-Making

- Each intersection shall be modeled as a reinforcement learning agent within a cooperative multi-agent formulation that is trained centrally and executed in a decentralized manner (CTDE).
- Each agent shall observe its local traffic state and select a signal-control action according to its learned policy, using only local observations at execution time.

5.1.5. Signal Phase Updates

- The system shall apply the agents' chosen actions to the simulator via TraCI, updating signal phases accordingly.
- The system shall respect minimum green and amber-time safety constraints when switching phases.

5.1.6. Emergency Vehicle Priority

- The system shall detect emergency vehicles approaching an intersection within a defined range.
- The responsible agent shall be able to adapt its phase selection to grant priority and clear the corresponding approach.
- The prioritization behavior shall be logged for later analysis of its impact on overall traffic.

5.1.7. Dashboard Reporting

- The system shall compute and display performance metrics including average waiting time, average queue length, throughput, and average travel time.
- The metrics shall be presented both in real time during a run and as end-of-run summaries.

5.1.8. Experiment Comparison

- The system shall store completed runs together with their configurations and results.
- The system shall allow the user to compare two or more experiments (for example, MARL versus fixed-time) side by side using tables and charts.

5.1.9. Secure Logging, Monitoring, and Resilience

- The system shall expose application and traffic-performance metrics through an instrumented endpoint that is scraped by Prometheus and visualized in Grafana dashboards.

- The system shall maintain structured, centralized logs of user actions, simulation runs, and agent decisions, and shall raise alerts when anomalous conditions are detected (for example, abnormal request rates, repeated failed authentications, or implausible traffic states).
- The system shall enforce API security controls, including authenticated access, input validation, and request rate limiting, on all simulation-control endpoints.
- The system shall support an attack-simulation mode in which falsified vehicle or sensor inputs, such as spoofed emergency-vehicle signals or perturbed observations, can be injected in order to evaluate the robustness of the learned policies.

5.2. Non-Functional Requirements

- **Usability:** The dashboard shall provide an intuitive interface with consistent visual encoding of signal states and interpretable charts.
- **Performance:** The simulator-agent loop and the streaming of state shall operate at a frame rate sufficient for smooth visualization, and action selection shall not bottleneck the simulation.
- **Scalability:** The architecture shall allow the number of intersections in the network to be increased without structural code changes, within the limits of available compute.
- **Security:** All simulation-control API endpoints shall require authentication, validate inputs, enforce request rate limiting, and communicate over HTTPS/TLS; secrets and tokens shall be handled following established security practices, and the running system shall be continuously monitored for anomalous activity.
- **Observability:** The system shall be instrumented so that its health, performance, and security-relevant events are measurable in real time through metrics, logs, and alerts.
- **Maintainability:** The codebase shall be modular, documented, and organized so that components can be modified independently.
- **Modularity:** The four-layer architecture shall enforce clear interfaces so that the RL algorithm or the road network can be swapped without affecting other layers.
- **Reproducibility:** Experiments shall be reproducible through fixed random seeds, version-controlled configurations, containerized environments, and averaging of results across multiple runs.

5.3. Risk Analysis

Table 5.1: Risk analysis with likelihood, impact, and mitigation.

Risk	Likelihood	Impact	Mitigation
MARL training instability or non-convergence	Medium	High	Use curriculum from single to multi-intersection, monitor learning curves, and fall back to VDN or tuned baselines if needed.
SUMO/TraCI integration failures	Medium	High	Build incremental integration tests, pin simulator versions, and keep mock interfaces for debugging.
Compute limits leading to long training times	Medium	High	Start with small networks, checkpoint runs, and schedule experiments to reuse trained policies.
Emergency-vehicle modeling inaccuracies	Low	Medium	Validate behavior in controlled scenarios and calibrate emergency routes and priorities in SUMO.
Dashboard-backend integration delays	Medium	Medium	Define stable API contracts early and use mock data to unblock UI work.
Security testing overhead or false positives	Low	Medium	Scope OWASP ZAP to project endpoints, run in isolation, and triage findings.
Reproducibility drift across environments	Medium	Medium	Fix seeds, version configs, and provide Docker Compose setup with locked dependencies.

5.4. Expected Deliverables

1. SUMO network and traffic scenario files with emergency vehicle definitions.
2. MARL environment wrapper and QMIX training pipeline with configuration files.
3. Emergency vehicle prioritization module integrated into the reward and action logic.
4. Baseline controllers (fixed-time and rule-based) and evaluation scripts.
5. FastAPI backend and web dashboard for control, monitoring, and experiment comparison.

6. Experiment results pack covering scenarios S1–S5 and ablation studies (tables, plots, and logs).
7. Reproducibility package including Docker Compose, run scripts, and final documentation.

6. Diagrams

6.1. Use Case Diagram

The use case diagram depicts the interactions between the actors and the system. The primary actor is the Researcher/Operator, whose use cases include configuring a simulation, starting and stopping runs, monitoring traffic state, training agents, triggering an emergency scenario, viewing metrics, and comparing experiments. A secondary actor, the MARL Agent, participates in use cases such as observing state, selecting an action, and updating the signal phase.

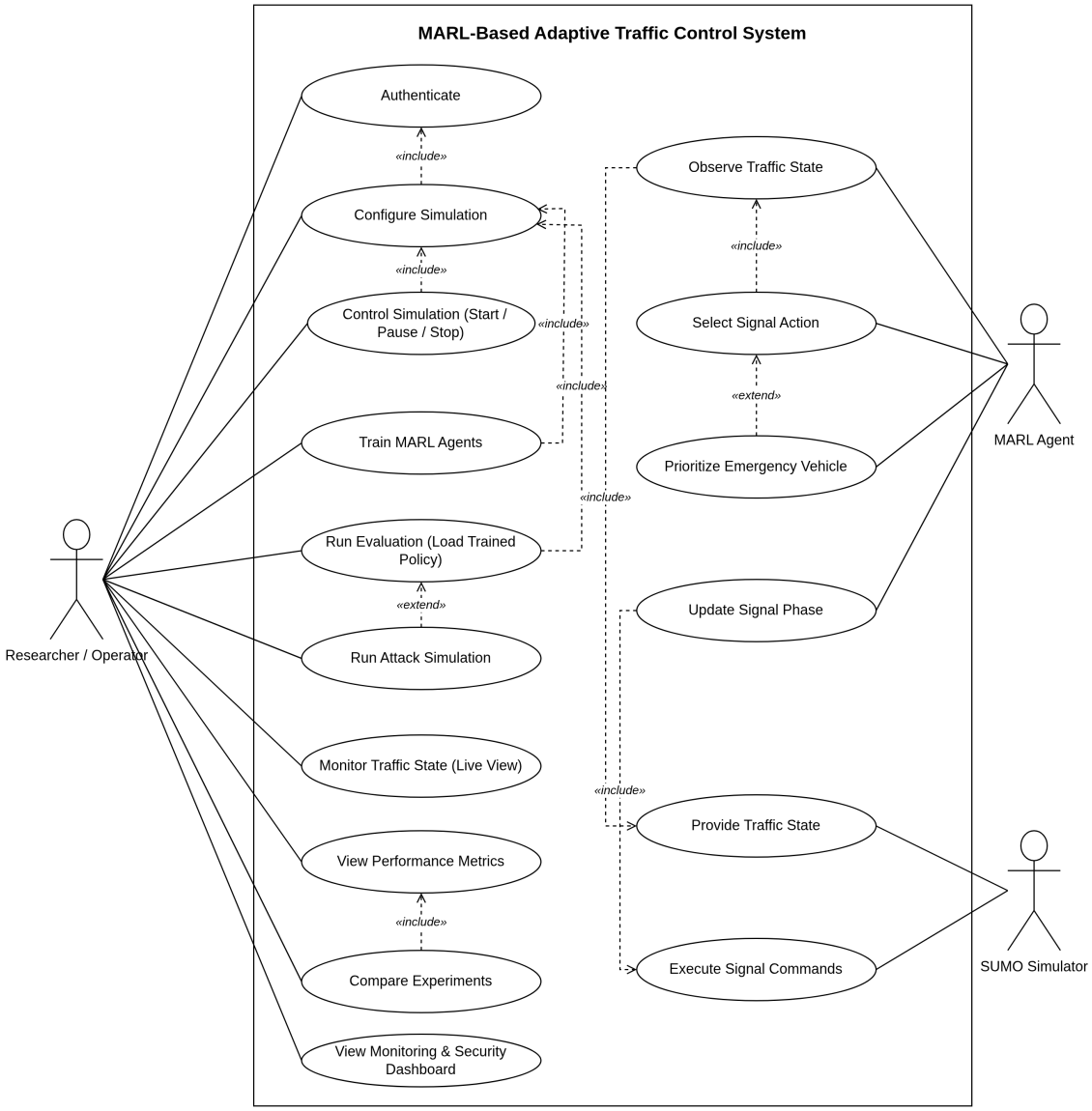


Figure 6.1: Use Case Diagram for the MARL Traffic Control System

6.2. Data Flow Diagram

6.2.1. DFD Level 0

The Level 0 (context) diagram represents the entire system as a single process situated between the external entities User and SUMO Simulator. The user supplies configuration and control commands and receives metrics and visualizations, while the simulator exchanges traffic state and signal-control commands with the system.

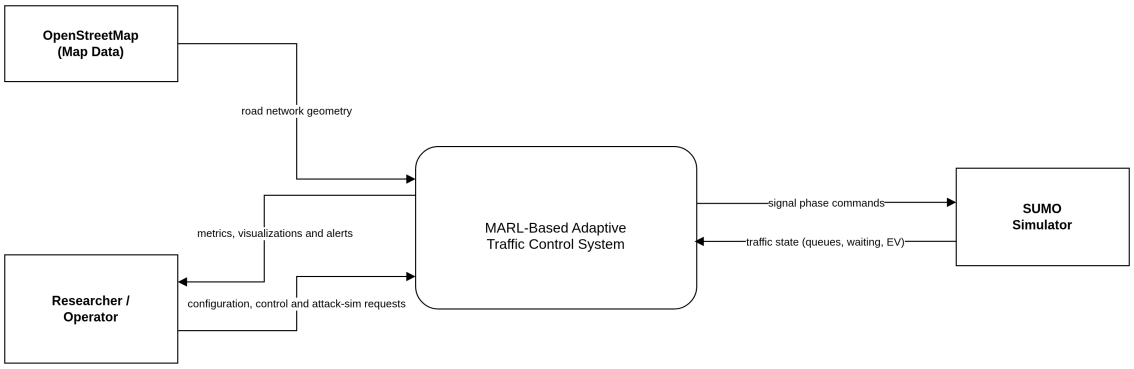


Figure 6.2: Data Flow Diagram Level 0 for the MARL Traffic Control System

6.2.2. DFD Level 1

The Level 1 diagram decomposes the system into its principal processes, namely simulation control, state extraction, agent decision-making, signal phase application, emergency prioritization, and metrics and reporting, together with data stores for learned policies, experiment configurations, and run results.

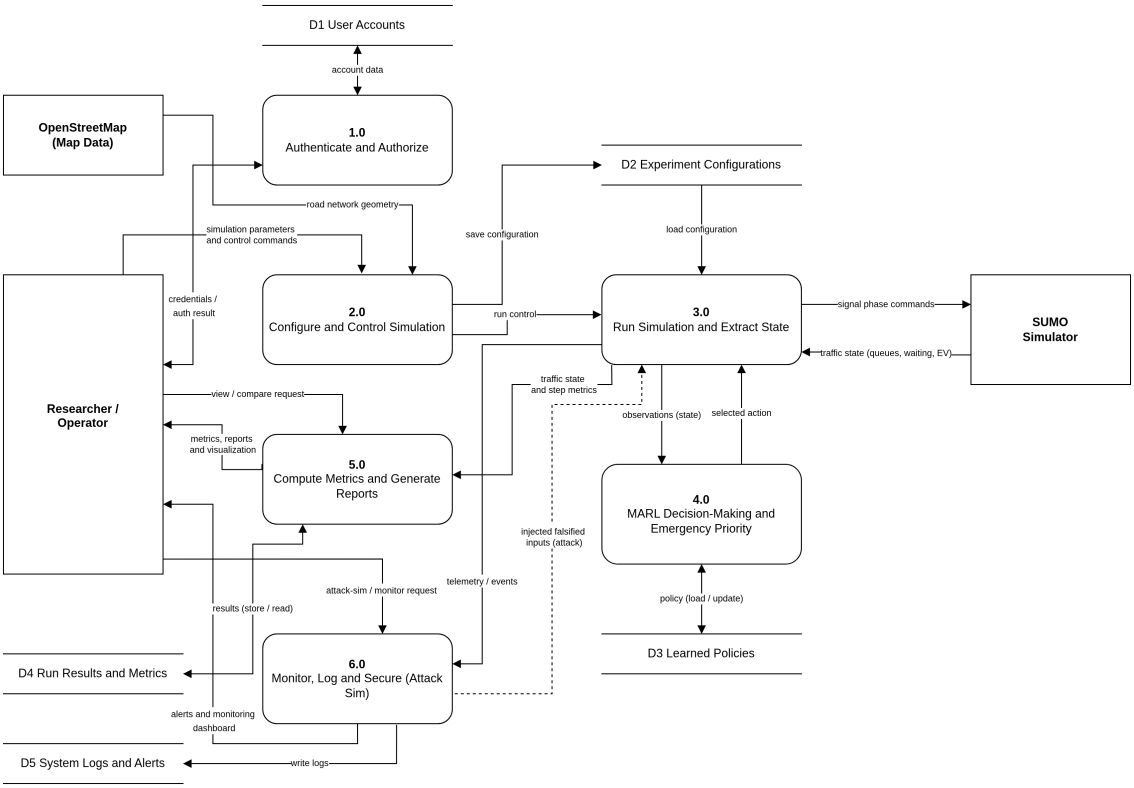


Figure 6.3: Data Flow Diagram Level 1 for the MARL Traffic Control System

6.3. System Architecture Diagram

The system architecture diagram illustrates the four-layer architecture, comprising the simulation layer (SUMO), the interface layer (TraCI/Python), the learning layer (Gymnasium/PettingZoo environment with PyTorch QMIX agents), and the application layer (FastAPI backend with a React and Tailwind CSS dashboard), together with the bidirectional communication paths between them. It also depicts the cross-cutting security and observability layer (authenticated APIs, structured logging, and the Prometheus/Grafana monitoring stack) that wraps the application layer.

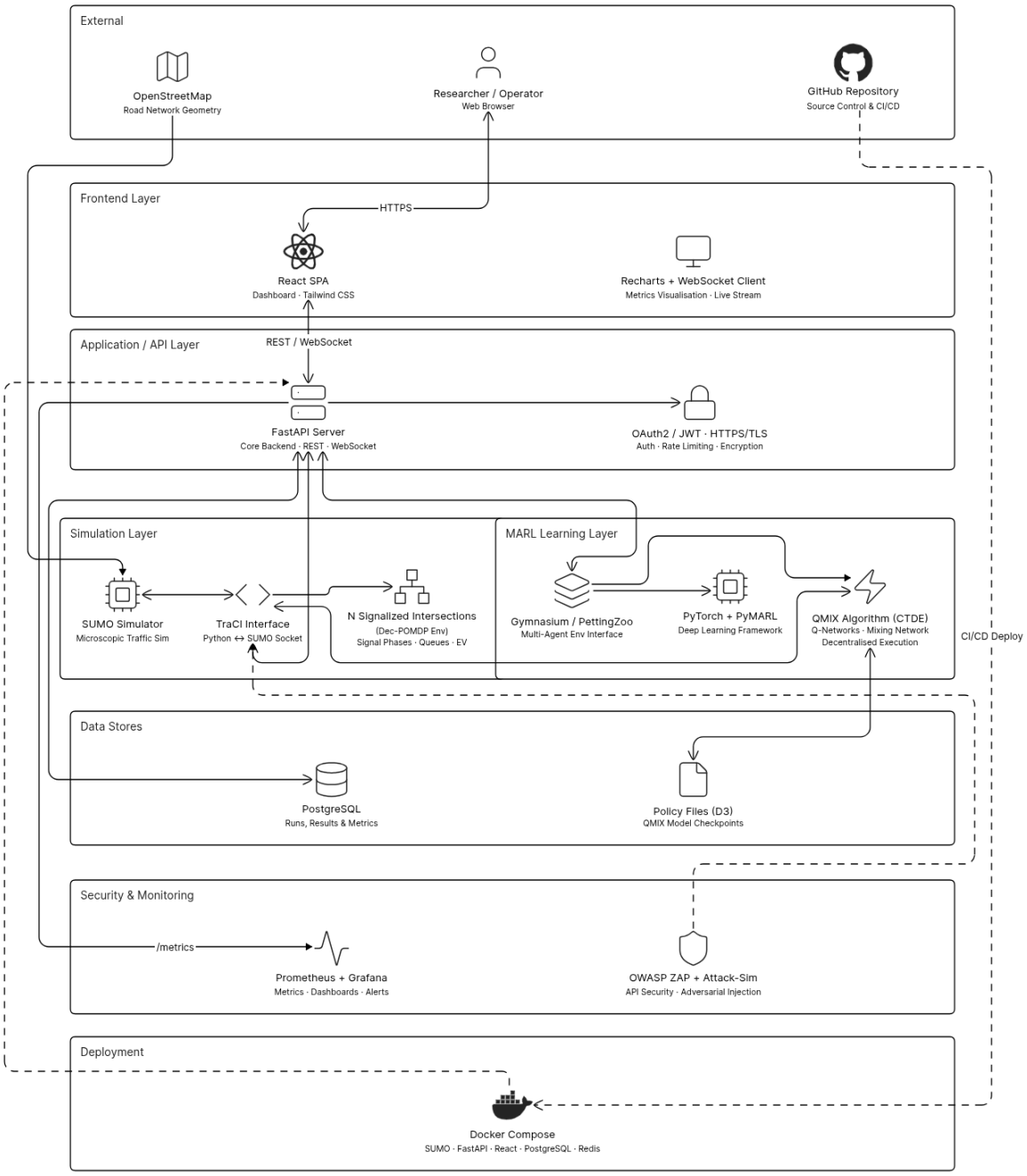


Figure 6.4: System Architecture Diagram

6.4. Gantt Chart

The Gantt chart presents the project schedule across its phases, namely requirements gathering, design, single-agent development, multi-agent development, emergency prioritization, dashboard integration, security and monitoring integration, testing and evaluation, and documentation, with overlapping bars reflecting the iterative nature of the methodology.

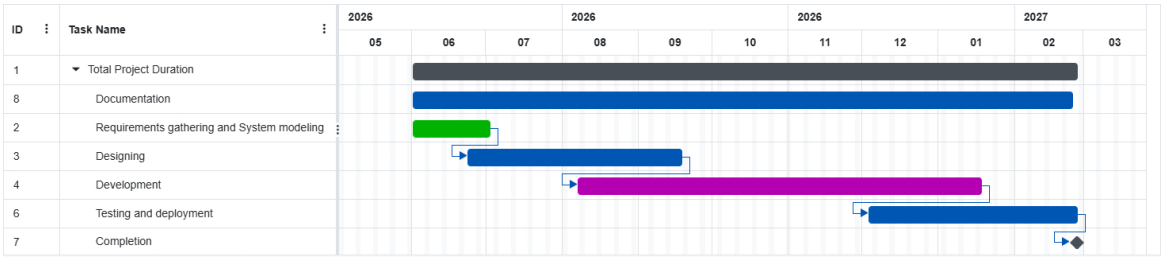


Figure 6.5: Gantt Chart

7. Tools and Technologies

The proposed system will be developed using modern, scalable, and well-supported open-source tools and technologies to ensure reproducibility, good performance, and an effective user experience. The key frameworks, programming languages, and tools used during the development phase are outlined below.

7.1. Frontend Infrastructure

- **HTML, CSS, and JavaScript:** Core web technologies used to structure, style, and add interactivity to the dashboard.
- **React:** A component-based JavaScript library used to build the interactive single-page dashboard application.
- **Tailwind CSS:** A utility-first CSS framework that enables a consistent, responsive, and modern visual design.
- **Charting Libraries (Recharts / Chart.js):** Used to render time-series and summary visualizations of the traffic performance metrics.

7.2. Backend Infrastructure

- **FastAPI:** A high-performance Python web framework that exposes REST endpoints for simulation control and uses WebSocket connections to stream real-time state and metrics.
- **Python:** The primary integration language across the backend, the simulator interface, and the learning components.
- **PostgreSQL / SQLite:** A relational database used to persist experiment configurations, run metadata, and aggregated performance metrics for later comparison.
- **Redis:** An in-memory store used for request rate limiting, session handling, and caching of streamed state where required.

7.3. Simulation Tools

- **SUMO (Simulation of Urban MObility):** An open-source microscopic traffic simulator used to model the road network, traffic demand, and emergency vehicle classes.
- **TraCI (Traffic Control Interface):** Provides the programmatic, step-by-step coupling between Python and SUMO for observation extraction and signal control.

7.4. AI/ML Tools

- **PyTorch:** The deep learning framework used to implement the agents' Q-networks and the QMIX mixing network.
- **QMIX / VDN (PyMARL or EPyMARL):** A multi-agent reinforcement learning framework providing implementations of value-decomposition algorithms (QMIX as the primary method, VDN as a baseline) under the CTDE paradigm.
- **Gymnasium / PettingZoo:** Provide the standardized single-agent and multi-agent reinforcement learning environment interfaces into which the SUMO environment is wrapped.
- **Stable-Baselines3:** Supplies reliable reference implementations used to train the single-agent RL baseline controller; the fixed-time and rule-based baselines are implemented directly, as they require no learning.
- **TensorBoard / Weights & Biases:** Used to track and visualize training curves (reward, loss, and evaluation metrics) across reinforcement learning experiments.
- **NumPy and Pandas:** Used for numerical computation and aggregation of evaluation metrics.

7.5. APIs and Data Sources

- **OpenStreetMap (OSM):** An open-source mapping service used to obtain realistic road network geometry, which SUMO's import tools convert into simulation networks.
- **Internal REST and WebSocket APIs:** The application's own programmatic interface, exposed by the FastAPI backend.

7.6. Hosting and Deployment

- **Docker:** Used to containerize the simulator, learning back-end, and web application, ensuring consistent and portable execution across environments.

7.7. Security and Monitoring Infrastructure

Because the system exposes a control API and learning-based controllers that are themselves a potential attack surface, a dedicated security and observability layer is included in the design [10].

- **Prometheus:** An open-source time-series monitoring system that periodically scrapes metrics (request counts, latencies, error rates, and traffic-performance indicators) exposed by an instrumented `/metrics` endpoint.
- **Grafana:** An open-source visualization and alerting platform used to build live dashboards over the Prometheus metrics and to trigger alerts when thresholds are breached.
- **OWASP ZAP:** An open-source dynamic application security testing tool used to probe the API for common vulnerabilities (injection, broken authentication, and misconfiguration).

7.8. Testing Tools

- **Pytest:** Used for unit and integration testing of the Python components.
- **SUMO Evaluation Runs:** Controlled experiments within SUMO are used to validate agent behavior, with metrics logged and analyzed using Pandas.
- **FastAPI Swagger:** Built-in interactive API documentation used to validate and test backend endpoints.
- **Locust:** Used for load and stress testing of the API to verify performance and resilience under concurrent requests.
- **OWASP ZAP:** Used for automated security regression testing of the API endpoints.

7.9. Version Control

- **Git:** A version control system used to track changes in the source code.

- **GitHub:** A collaborative platform used for source code hosting, code review, and issue tracking.

Other tools and technologies may be added in the future as required during the design and development of the system.

8. Conclusion

This proposal has outlined the design of a simulation-based research prototype for adaptive urban traffic signal control using Multi-Agent Reinforcement Learning. By modeling each signalized intersection as an autonomous learning agent within the SUMO simulator, training these agents with the QMIX value-decomposition algorithm under the centralized-training, decentralized-execution paradigm, coupling them to the simulator through the TraCI interface, and exposing the whole system through an interactive web dashboard, the project aims to demonstrate that learning-based, coordinated control can outperform classical fixed-time and rule-based strategies on standard traffic performance metrics.

The anticipated impact of the system is fourfold. First, by adapting signal timings to real-time demand and coordinating across intersections, the controller is expected to reduce congestion, evidenced by lower average waiting times, shorter queue lengths, and higher throughput relative to fixed-time and rule-based baselines. Second, the multi-agent formulation is expected to yield improved network-wide coordination, with gains that grow as traffic load increases, consistent with established findings in the literature. Third, the emergency vehicle prioritization mechanism is expected to reduce the delay experienced by emergency vehicles while keeping the disruption to ordinary traffic measurable and bounded. Fourth, the integrated security and observability layer, comprising API hardening, Prometheus/Grafana monitoring, and an attack-simulation facility, is expected to demonstrate that the controller can be operated securely and that its robustness to adversarial inputs can be measured rather than assumed.

A limitation is the sim-to-real gap: SUMO abstracts driver behavior, sensing noise, and compliance, so performance may not transfer directly to real deployments. The project therefore treats results as evidence of feasibility and relative performance in simulation, and future work would require calibration with real data, hardware-in-the-loop testing, and staged field trials.

Beyond these performance outcomes, the project's research value lies in providing a coherent, reproducible, and extensible platform that integrates multi-agent learning, emergency prioritization, secure and observable operation, and experiment management. The scope is deliberately kept practical and appropriate for a major project, namely a simulated network of a modest number of intersections rather than a large-scale production deployment. Nevertheless, the architecture and findings are intended to lay a foundation that could be extended in future work toward larger networks, more realistic demand models, and, ultimately, the controlled study of real-world deployment.

References

[1] P. A. Lopez, M. Behrisch, L. Bieker-Walz, J. Erdmann, Y.-P. Flötteröd, R. Hilbrich, L. Lücken, J. Rummel, P. Wagner, and E. Wießner, "Microscopic traffic simulation using sumo," in *Proceedings of the 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, (Maui, HI, USA), pp. 2575–2582, 2018.

[2] B. Abdulhai, R. Pringle, and G. J. Karakoulas, "Reinforcement learning for true adaptive traffic signal control," *Journal of Transportation Engineering*, vol. 129, no. 3, pp. 278–285, 2003.

[3] H. Wei, G. Zheng, H. Yao, and Z. Li, "Intellilight: A reinforcement learning approach for intelligent traffic light control," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, (London, UK), pp. 2496–2505, 2018.

[4] M. A. Wiering, "Multi-agent reinforcement learning for traffic light control," in *Proceedings of the 17th International Conference on Machine Learning (ICML)*, (Stanford, CA, USA), pp. 1151–1158, 2000.

[5] L. Buşoniu, R. Babuška, and B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 38, no. 2, pp. 156–172, 2008.

[6] R. Donatus, K. Ter, O.-O. Ajayi, and D. Udekwe, "Multi-agent reinforcement learning in intelligent transportation systems: A comprehensive survey," *arXiv preprint arXiv:2508.20315*, 2025.

[7] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, "Value-decomposition networks for cooperative multi-agent learning based on team reward," in *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pp. 2085–2087, 2018.

[8] T. Rashid, M. Samvelyan, C. S. de Witt, G. Farquhar, J. Foerster, and S. Whiteson, "QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, pp. 4295–4304, 2018.

[9] T. Chu, J. Wang, L. Codecà, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 3, pp. 1086–1095, 2020.

[10] A. Haydari, M. Zhang, and C.-N. Chuah, "Adversarial attacks and defense in deep reinforcement learning (DRL)-based traffic signal controllers," *IEEE Open Journal of Intelligent Transportation Systems*, vol. 2, pp. 402–416, 2021.